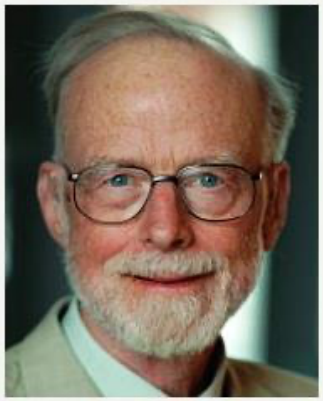
排序
快速排序：轴点

分而治之

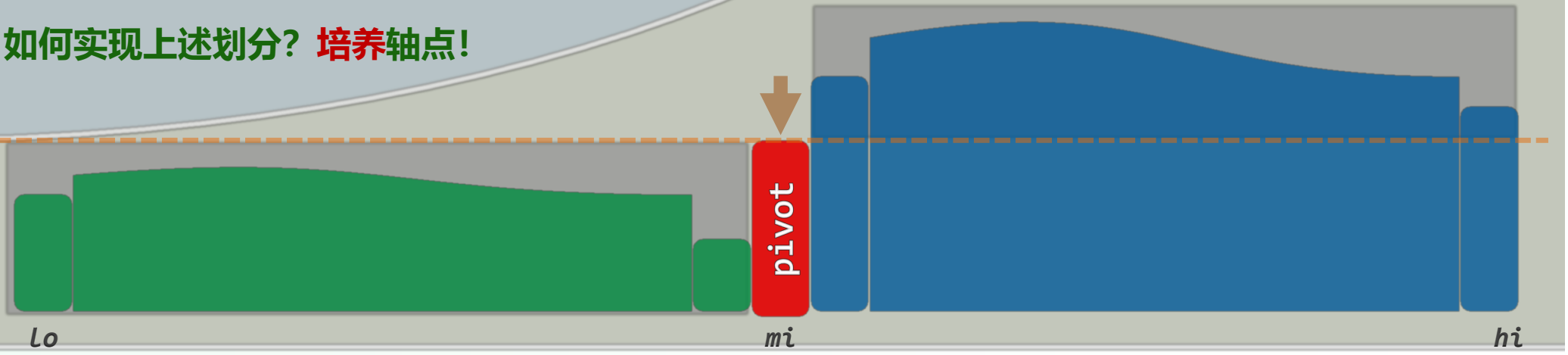
- pivot：左侧/右侧的元素，均不比它更大/更小
- 以轴点为界，自然划分：max([0, mi)) ≤ min((mi, hi))
- 前缀、后缀各自（递归）排序之后，原序列自然有序

$$sorted(S) = sorted(S_L) + sorted(S_R)$$

- mergesort难点在于合，而quicksort在于分
- 如何实现上述划分？培养轴点！

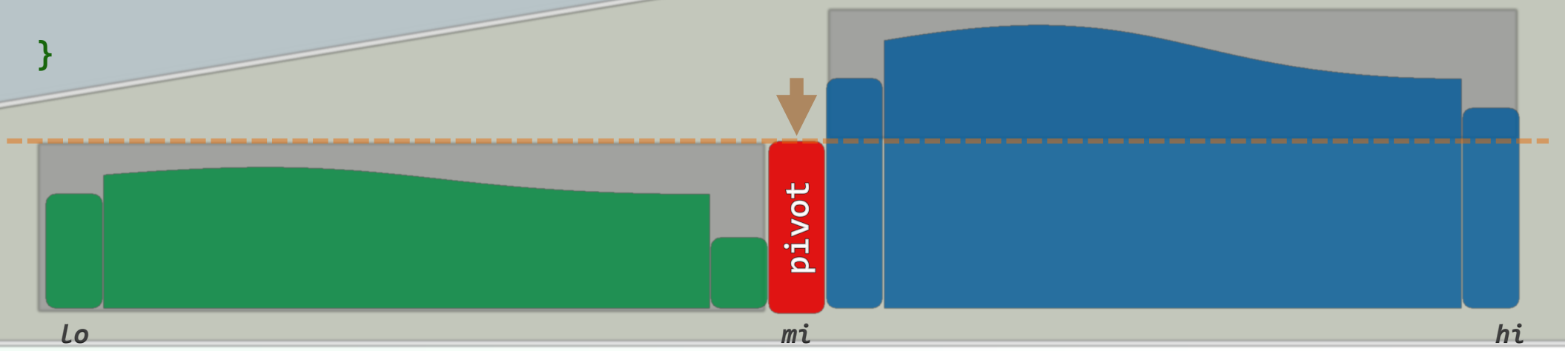


C. A. R. Hoare
(1934 ~)
Turing Award, 1980



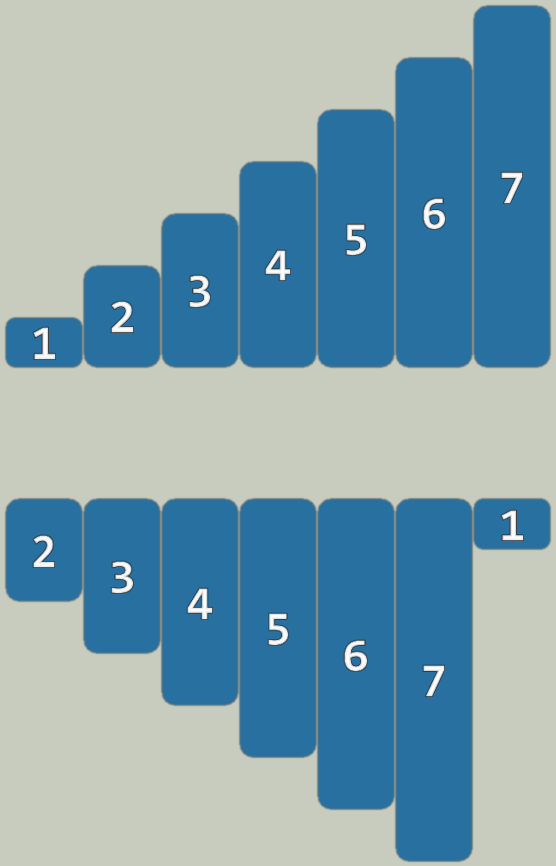
快速排序

```
❖ template <typename T> void Vector<T>::quickSort( Rank lo, Rank hi ) {
     if ( hi - lo < 2 ) return;
     Rank mi = partition( lo, hi ); //能否足够高效?
     quickSort( lo, mi );
     quickSort( mi + 1, hi );
  }
```



轴点

- 必要条件：轴点必定已然就位 //尽管反之不然
- 特别地：在有序序列中，所有元素皆为轴点
  反之亦然
- 快速排序：就是将所有元素逐个转换为轴点的过程
- 坏消息：在原始序列中，轴点未必存在...
- derangement：任何元素都不在原位
  比如，顺序序列循环移位
- 好消息：不需很多交换，即可使任一元素转为轴点
- 问题：如何交换？成本多高？



排序
快速排序：快速划分：LUG版

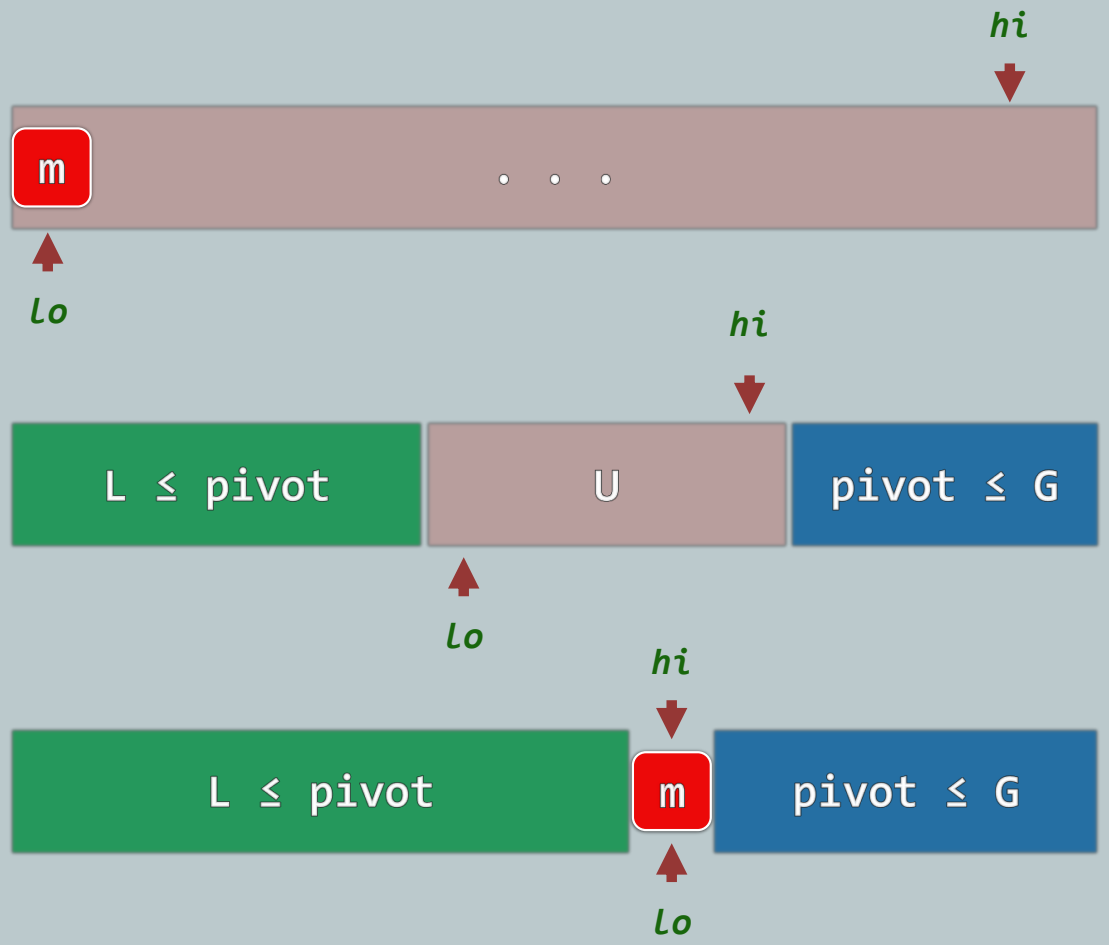
减而治之，相向而行

- 任取一个候选者（如[0]）
- L + U + G
- 交替地向内移动lo和hi
- 逐个检查当前元素：

  若更小/大，则转移归入L/G
- 当lo = hi时，只需

  将候选者嵌入于L、G之间，即成轴点！
- 各元素最多移动一次（候选者两次）

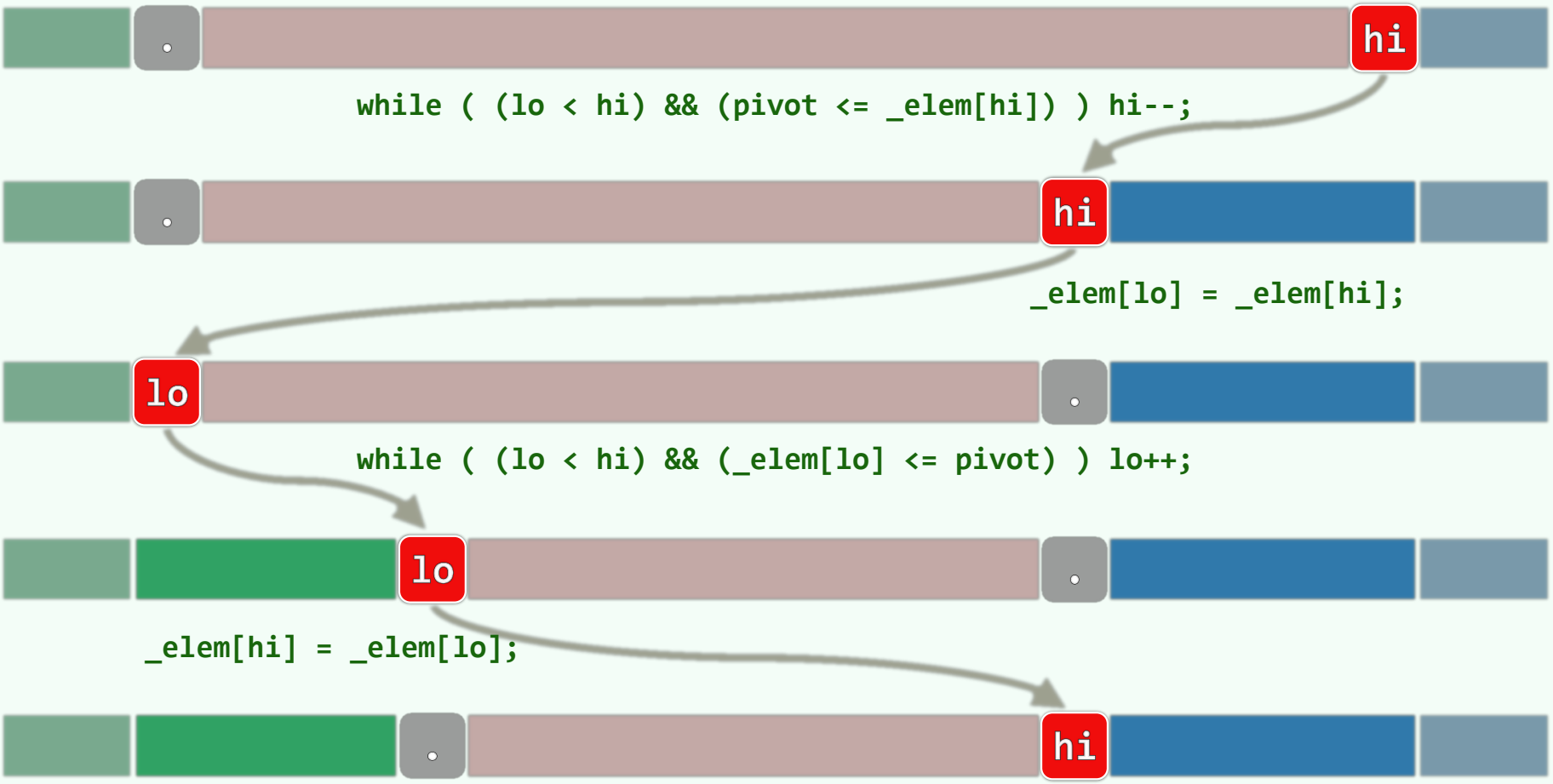
  ——累计O(n)时间、O(1)辅助空间



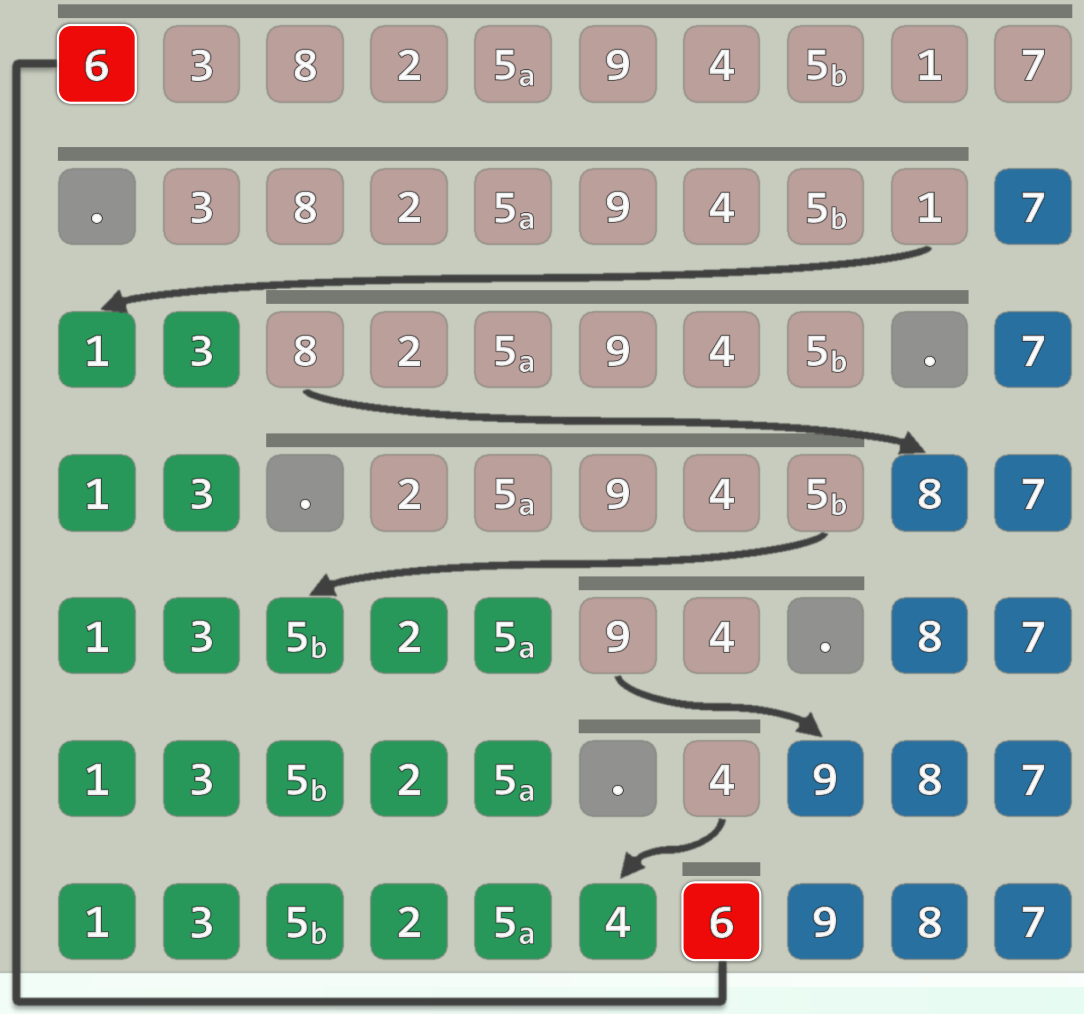
快速划分：LUG版

```
template <typename T> Rank Vector<T>::partition( Rank lo, Rank hi ) { //[lo, hi)
   swap( _elem[ lo ], _elem[ lo + rand() % ( hi - lo ) ] ); //随机交换
   hi--; T pivot = _elem[ lo ]; //经以上交换，等效于随机选取候选轴点
   while ( lo < hi ) { //从两端交替地向中间扫描，彼此靠拢
      while ( lo < hi  &&  pivot <= _elem[ hi ] ) hi--; //向左拓展G
      _elem[ lo ] = _elem[ hi ]; //凡小于轴点者，皆归入L
      while ( lo < hi  &&  _elem[ lo ] <= pivot ) lo++; //向右拓展L
      _elem[ hi ] = _elem[ lo ]; //凡大于轴点者，皆归入G
   } //assert: lo == hi
   _elem[ lo ] = pivot; return lo; //候选轴点归位；返回其秩
}
```

不变性 + 单调性：L ≤ pivot ≤ G； U = [lo, hi]中，[lo]和[hi]交替空闲



实例



- ❖ 线性时间
  - 尽管lo、hi交替移动
  - 累计移动距离不过$O(n)$
- ❖ in-place
  - 只需$O(1)$附加空间
- ❖ unstable
  - lo/hi的移动方向相反
  - 左/右侧的大/小重复元素可能前/后颠倒

排序
快速排序：比较次数

时间性能 + 随机

❖ 最好情况：每次划分都（接近）平均，轴点总是（接近）中央 —— 到达下界！

$$T(n) = 2 \cdot T(\frac{n-1}{2}) + \mathcal{O}(n) = \mathcal{O}(n \cdot \log n)$$

❖ 最坏情况：每次划分都极不均衡（比如，轴点总是最小/大元素） —— 与起泡排序相当！

$$T(n) = T(n-1) + T(0) + \mathcal{O}(n) = \mathcal{O}(n^2)$$

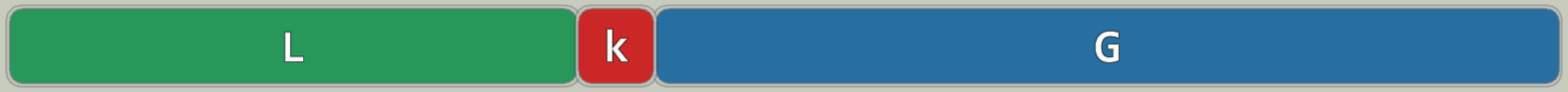
❖ 采用随机选取（Randomization）、三者取中（Sampling）之类的策略

只能降低最坏情况的概率，而无法杜绝 —— 既如此，为何还称作快速排序?

递推分析（1/2）

❖ 记期望的比较次数为 $T(n)$：　$T(1) = 0,\ T(2) = 1,\ \ldots$

❖ 可以证明：$T(n) = \mathcal{O}(n \log n)$　...



$$T(n) \ = \ (n-1) + \frac{1}{n} \cdot \sum_{k=0}^{n-1} [T(k) + T(n-k-1)] \ = \ (n-1) + \frac{2}{n} \cdot \sum_{k=0}^{n-1} T(k)$$

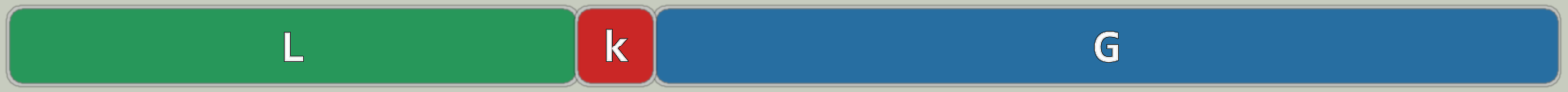
$$n \cdot T(n) \ = \ n \cdot (n-1) \ + \ 2 \times \sum_{k=0}^{n-1} T(k)$$

$$(n-1) \cdot T(n-1) \ = \ (n-1) \cdot (n-2) \ + \ 2 \times \sum_{k=0}^{n-2} T(k)$$

递推分析（2/2）

$$n \cdot T(n) - (n-1) \cdot T(n-1) = 2 \cdot (n-1) + 2 \times T(n-1)$$

$$n \cdot T(n) - (n+1) \cdot T(n-1) = 2 \cdot (n-1)$$



$$\frac{T(n)}{n+1} - \frac{T(n-1)}{n} = \frac{4}{n+1} - \frac{2}{n}$$

$$\frac{T(n)}{n+1} = \frac{T(n)}{n+1} - \frac{T(1)}{2} = 4 \cdot \sum_{k=2}^{n} \frac{1}{k+1} - 2 \cdot \sum_{k=2}^{n} \frac{1}{k} = 2 \cdot \sum_{k=1}^{n+1} \frac{1}{k} + \frac{2}{n+1} - 4 \approx 2 \cdot \ln n$$

$$T(n) \approx 2 \cdot n \cdot \ln n = (2 \cdot \ln 2) \cdot n \log n \approx 1.386 \cdot n \log n$$

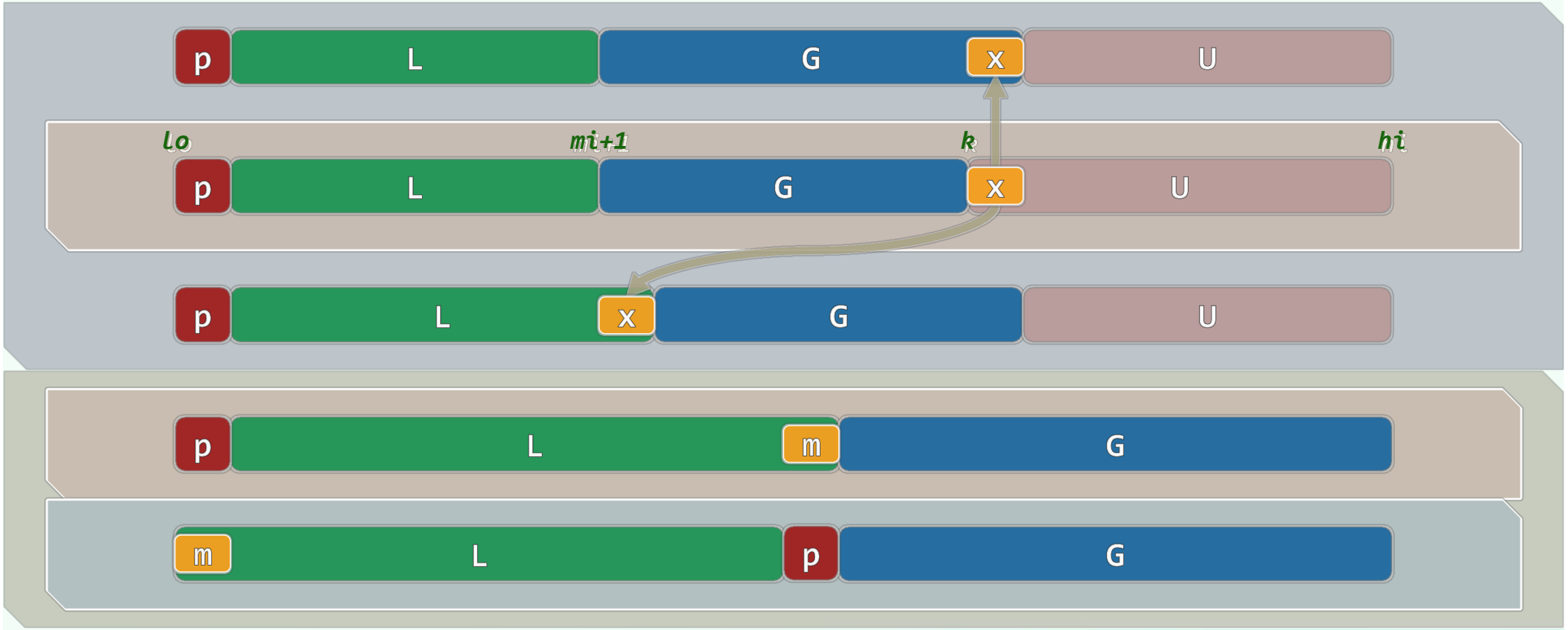
对比

	#compare	#move (对实际性能影响更大)	in-placed
Quicksort	平均 $O(1.386*n\log n)$ 且高概率接近	平均不超过 $O(1.386*n\log n)$ 且实际更少	✓
Mergesort	严格 $O(1.00*n\log n)$	严格 $O(1.00*n\log n)$ 实际往往加倍	✗

排序
快速排序：快速划分：LGU版

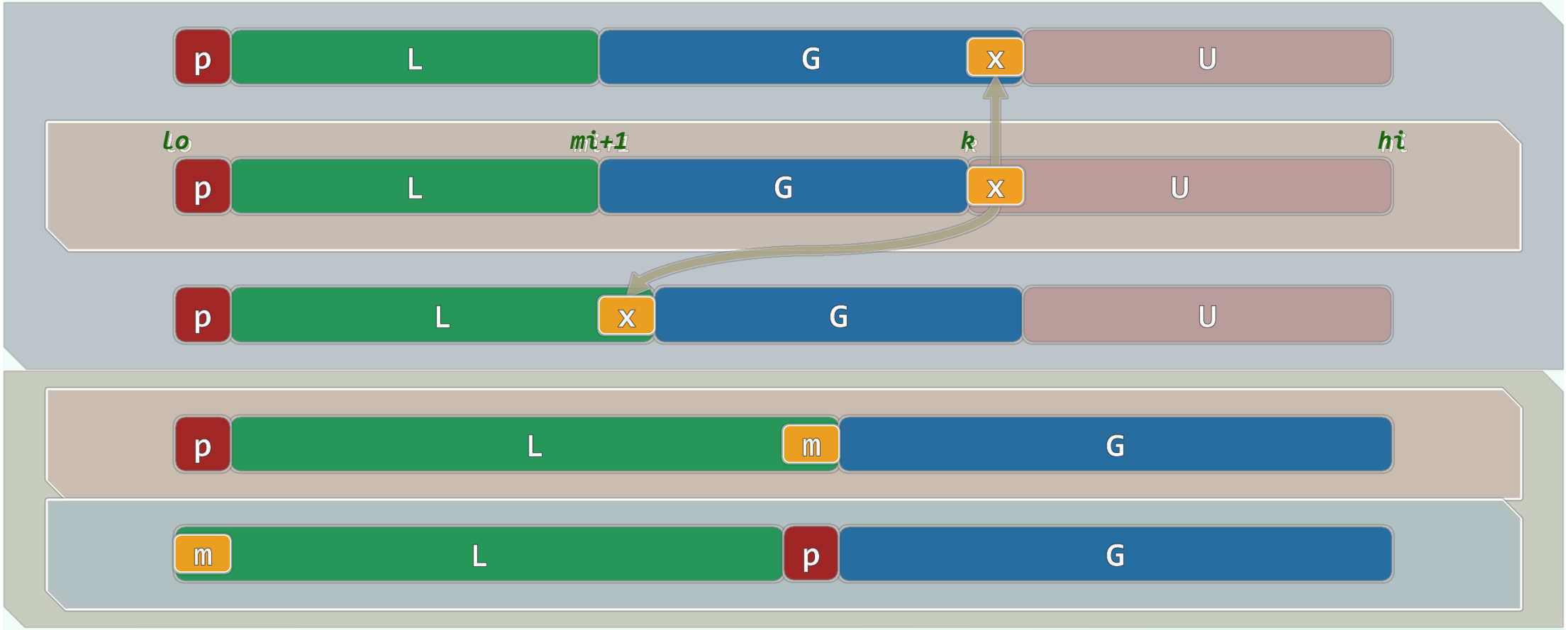
不变性： $S = [lo, hi) = [lo] + (lo, mi] + (mi, k) + [k, hi) = pivot + L + G + U$

$$L < pivot \leq G$$



单调性：[k]不小于轴点 ? 直接G拓展 : G滚动后移，L拓展

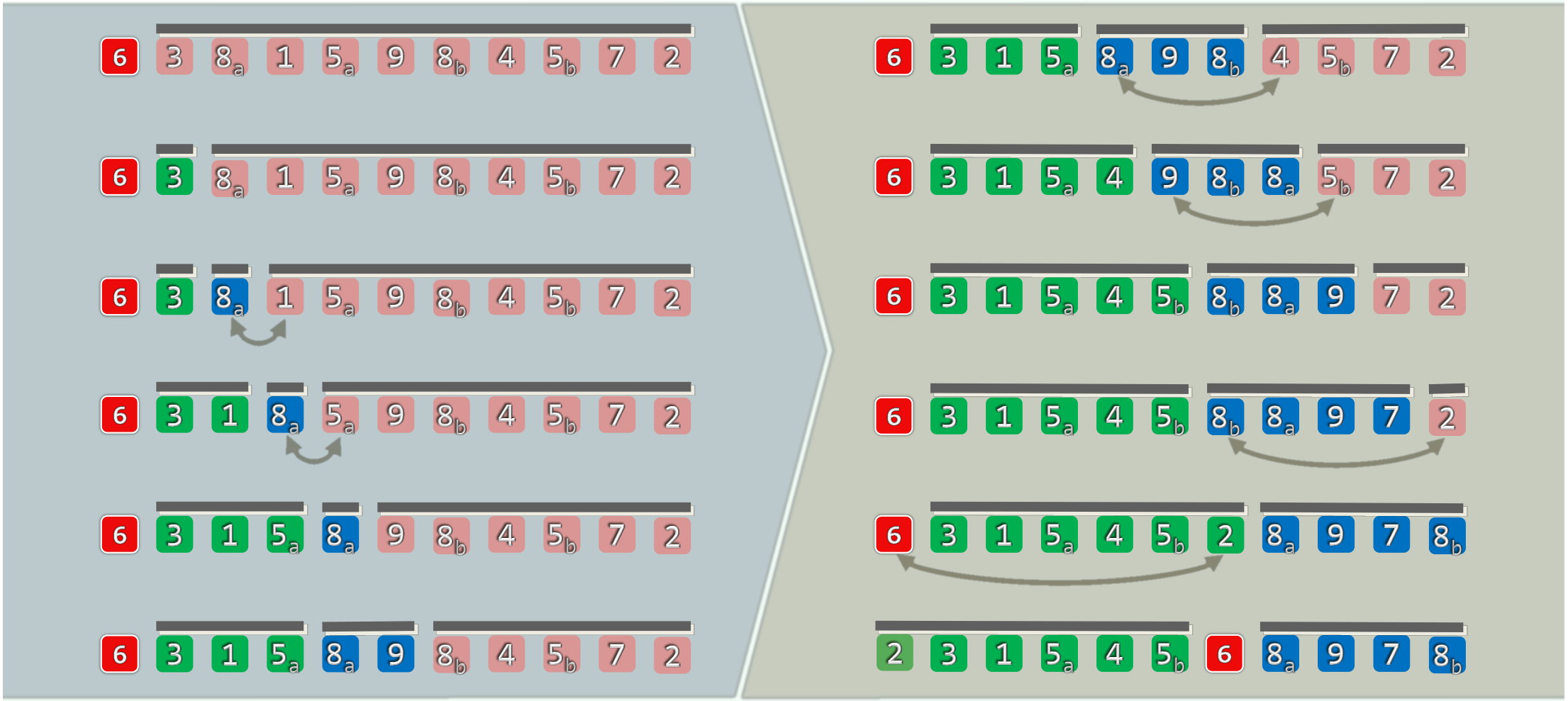
```
pivot <= S[ k ]  ?  k++  :  swap( S[ ++mi ], S[ k++ ] )
```



快速划分：LGU版

```
template <typename T> Rank Vector<T>::partition( Rank lo, Rank hi ) { //[lo, hi)
   swap( _elem[ lo ], _elem[ lo + rand() % ( hi - lo ) ] ); //随机交换
   T pivot = _elem[ lo ]; int mi = lo;
   for ( Rank k = lo + 1; k < hi; k++ ) //自左向右考查每个[k]
      if ( _elem[ k ] < pivot ) //若[k]小于轴点，则将其
         swap( _elem[ ++mi ], _elem[ k ]); //与[mi]交换，L向右扩展
   swap( _elem[ lo ], _elem[ mi ] ); //候选轴点归位（从而名副其实）
   return mi; //返回轴点的秩
}
```

实例

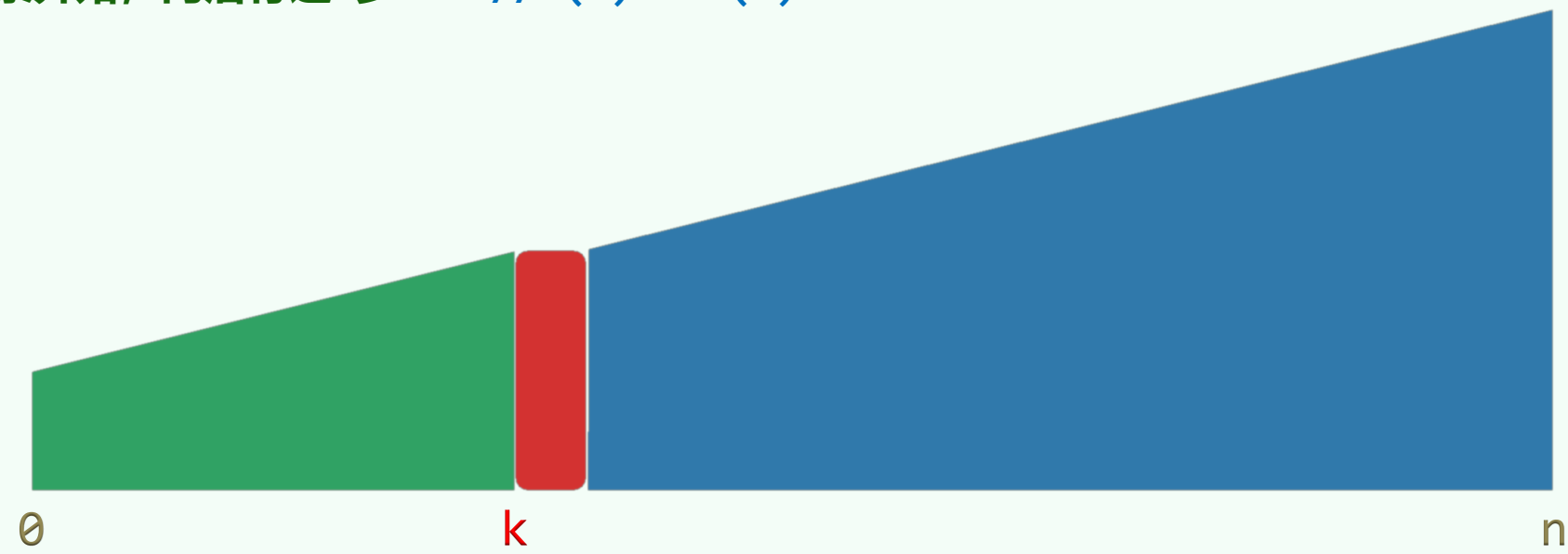

6 3 8a 1 5a 9 8b 4 5b 7 2
6 3 8a 1 5a 9 8b 4 5b 7 2
6 3 8a 1 5a 9 8b 4 5b 7 2
6 3 1 8a 5a 9 8b 4 5b 7 2
6 3 1 5a 8a 9 8b 4 5b 7 2
6 3 1 5a 8a 9 8b 4 5b 7 2
6 3 1 5a 8a 9 8b 4 5b 7 2
6 3 1 5a 4 9 8b 8a 5b 7 2
6 3 1 5a 4 5b 8b 8a 9 7 2
6 3 1 5a 4 5b 8b 8a 9 7 2
6 3 1 5a 4 5b 2 8a 9 7 8b
2 3 1 5a 4 5b 6 8a 9 7 8b

排序
选取：QuickSelect

尝试：蛮力

❖ 对A排序　　//O(nlogn)

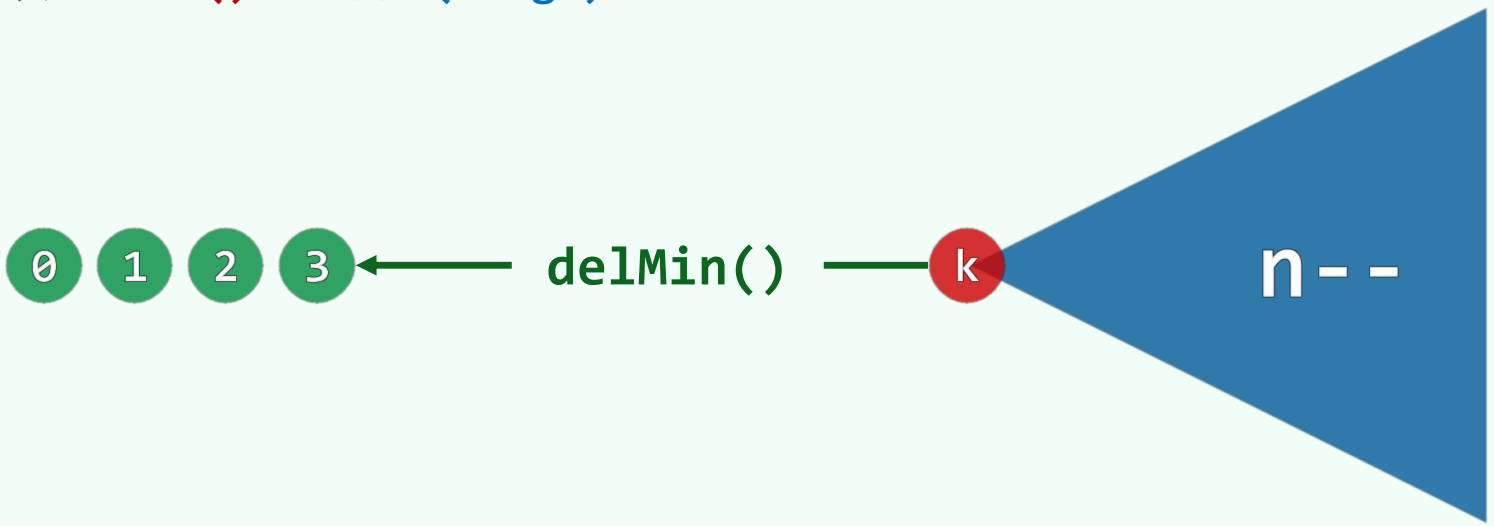
从首元素开始，向后行进k步　　//O(k) = O(n)



尝试：堆（A）

❖ 将所有元素组织为小顶堆　　//O(n)

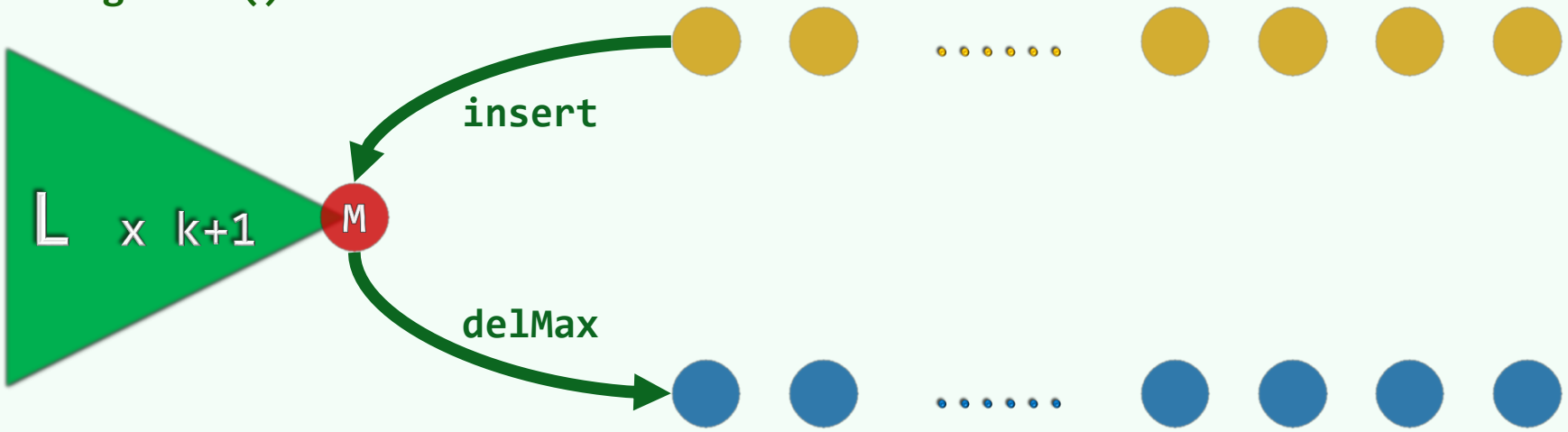
连续调用k+1次delMin()　　//O(klogn)



尝试：堆（B）

- L = heapify(A[0, k]) //任选 k+1 个元素，组织为大顶堆：O(k)
- for each i in (k, n) //O(n - k)
  - L.insert(A[i]) //O(logk)
  - L.delMax() //O(logk)

return L.getMax()



尝试：堆（C）

❖ 将输入任意划分为规模为k、n-k的子集

分别组织为大、小顶堆

//O(k + (n-k)) = O(n)

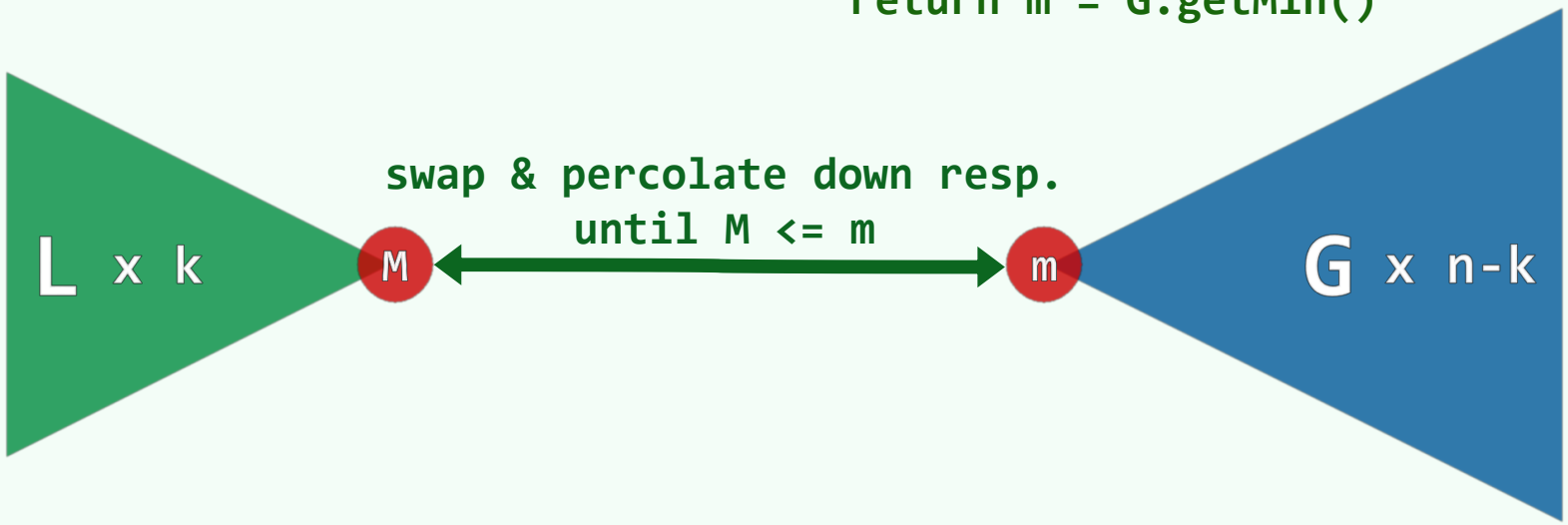
❖ while (m < M) //O(min(k, n - k))

swap(m, M)

L.percolateDown() //O(logk)

G.percolateDown() //O(log(n - k))

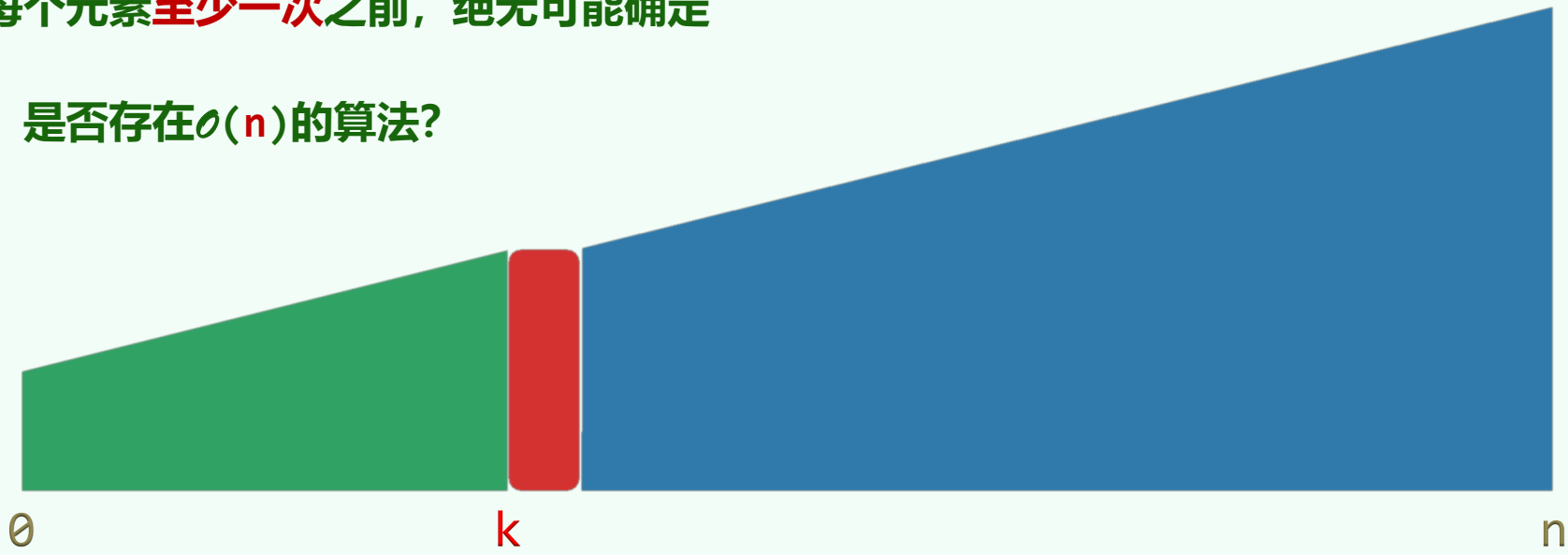
return m = G.getMin()



下界与最优

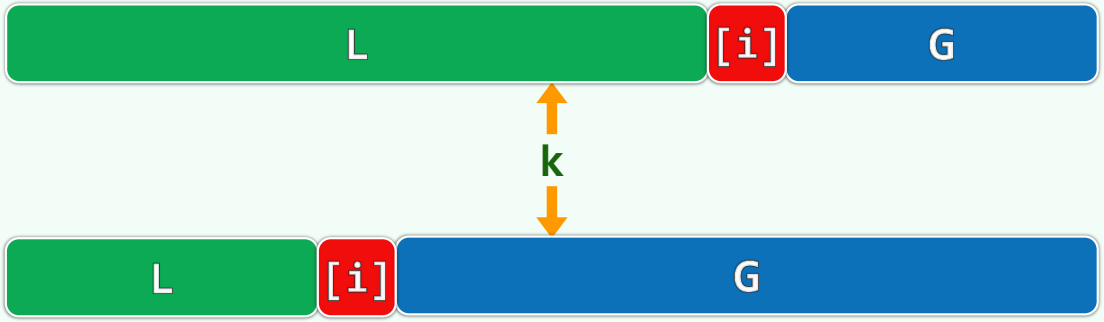
- 是否存在更快的算法？当然，最快也不至于快过$\Omega(n)$！
- 所谓第k小，是相对于序列整体而言，所以...

  在访问每个元素至少一次之前，绝无可能确定
- 反过来，是否存在$O(n)$的算法？



quickSelect()

```
template <typename T> void quickSelect( Vector<T> & A, Rank k ) {
   for ( Rank lo = 0, hi = A.size() - 1; lo < hi;  ) {
      Rank i = lo, j = hi; T pivot = A[lo]; //大胆猜测
      while ( i < j ) { //小心求证: O(hi - lo + 1) = O(n)
         while ( i < j  &&  pivot <= A[j] ) j--;  A[i] = A[j];
         while ( i < j  &&  A[i] <= pivot ) i++;  A[j] = A[i];
      } //assert: quit with i == j
      A[i] = pivot;
      if ( k <= i ) hi = i - 1;
      if ( i <= k ) lo = i + 1;
   } //A[k] is now a pivot
}
```



期望性能

❖ 记期望的比较次数为 $T(n)$

$$T(1) = 0,\ T(2) = 1,\ \ldots$$

❖ 可以证明：$T(n) = \mathcal{O}(n)$ ...

$$T(n) \ = \ (n-1) \ + \ \frac{1}{n} \times \sum_{k=0}^{n-1} \max\{T(k), T(n-k-1)\} \ \leq \ (n-1) \ + \ \frac{2}{n} \times \sum_{k=n/2}^{n-1} T(k)$$

❖ 事实上，不难验证：$T(n) < 4 \cdot n$ ...

$$T(n) \ \leq \ (n-1) \ + \ \frac{2}{n} \times \sum_{k=n/2}^{n-1} 4k \ \leq \ (n-1) + 3n \ < \ 4n$$

排序
希尔排序：框架+实例

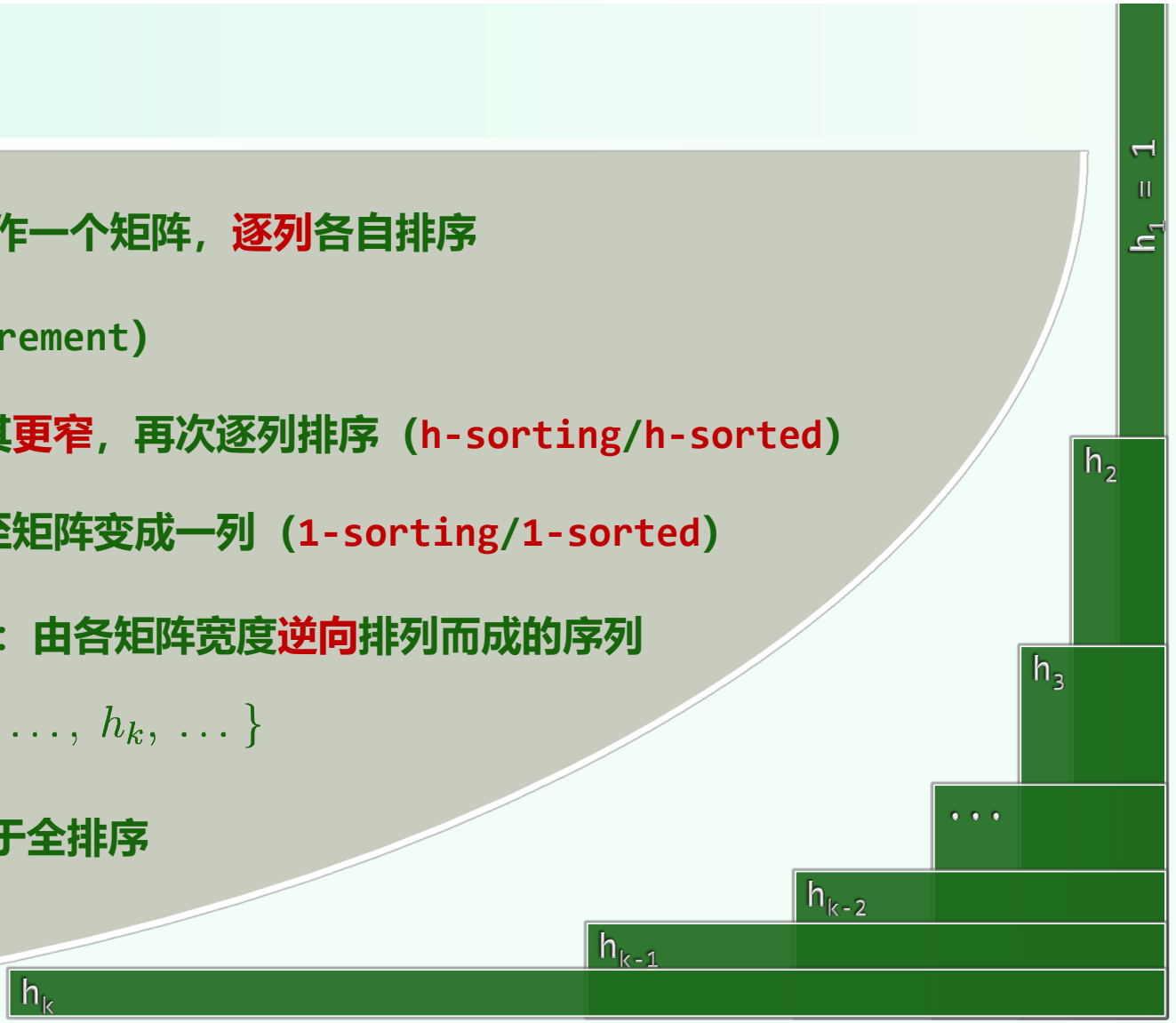
Shellsort

- D. L. Shell：将整个序列视作一个矩阵，逐列各自排序
- 递减增量（diminishing increment）
  - 由粗到细：重排矩阵，使其更窄，再次逐列排序（h-sorting/h-sorted）
  - 逐步求精：如此往复，直至矩阵变成一列（1-sorting/1-sorted）
- 步长序列（step sequence）：由各矩阵宽度逆向排列而成的序列

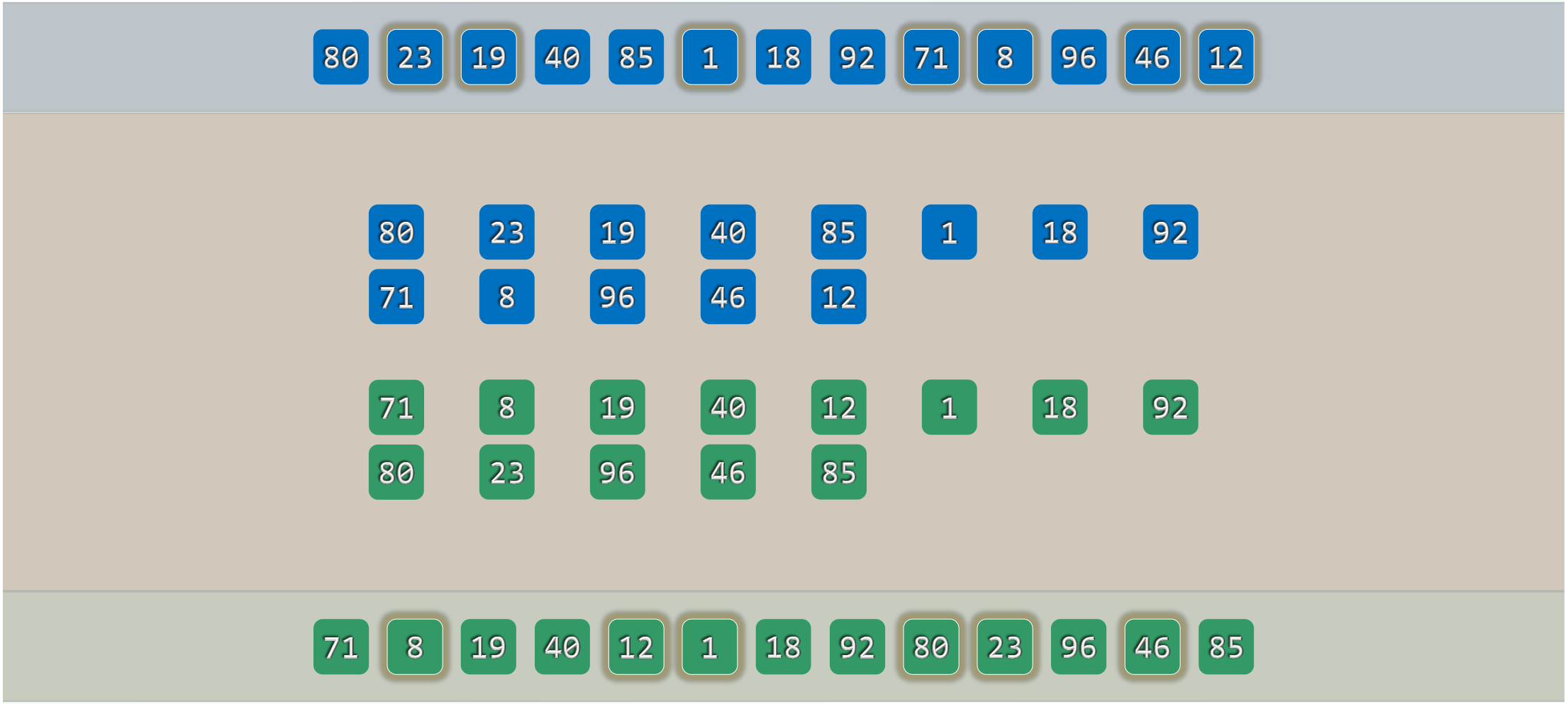
$$\mathcal{H} = \{ h_1 = 1, h_2, h_3, \ldots, h_k, \ldots \}$$

- 正确性：最后一次迭代，等同于全排序

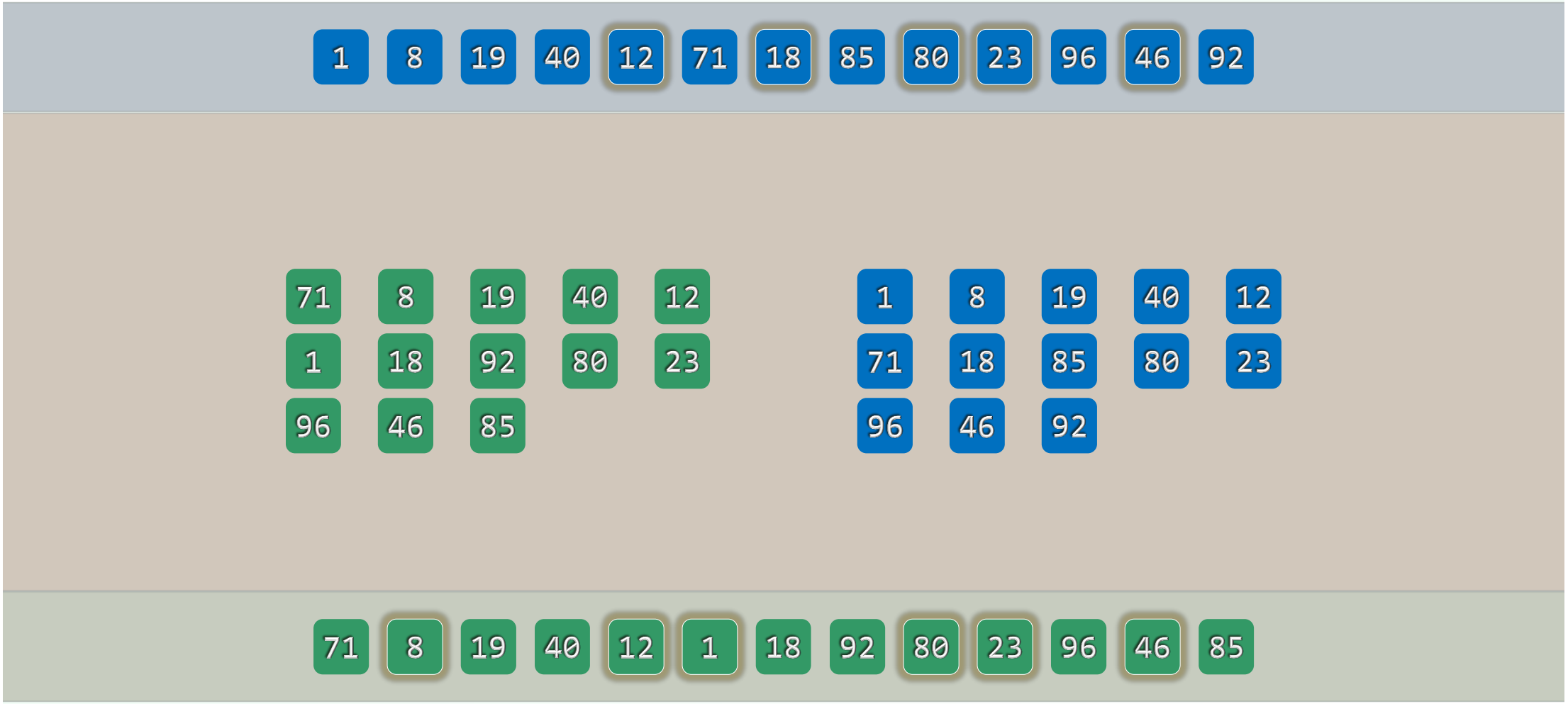
1-sorted = ordered



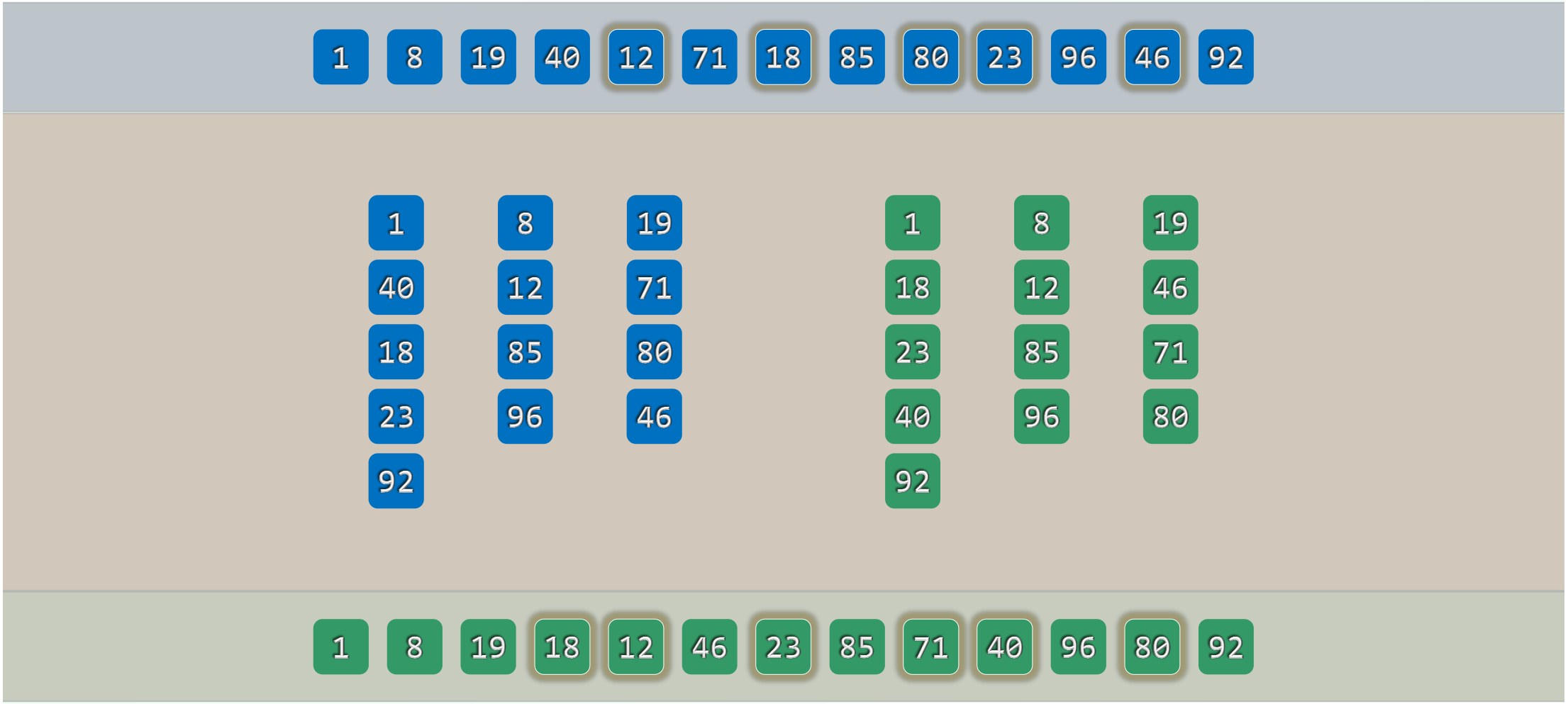
实例：$h_5 = 8$


80 23 19 40 85 1 18 92 71 8 96 46 12
80 23 19 40 85 1 18 92
71 8 96 46 12
71 8 19 40 12 1 18 92
80 23 96 46 85
71 8 19 40 12 1 18 92 80 23 96 46 85

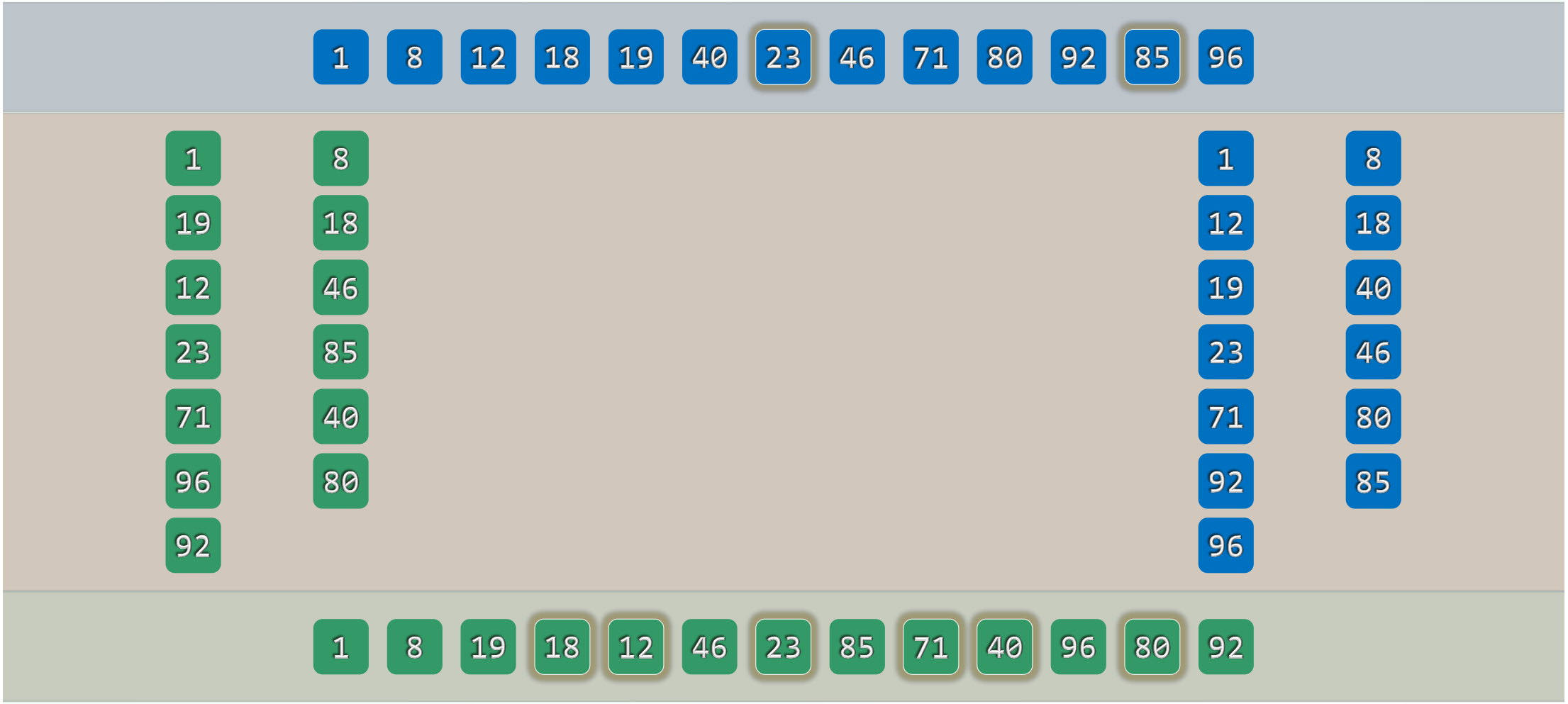
实例：$h_4 = 5$


1 8 19 40 12 71 18 85 80 23 96 46 92
71 8 19 40 12
1 18 92 80 23
96 46 85
1 8 19 40 12
71 18 85 80 23
96 46 92
71 8 19 40 12 1 18 92 80 23 96 46 85

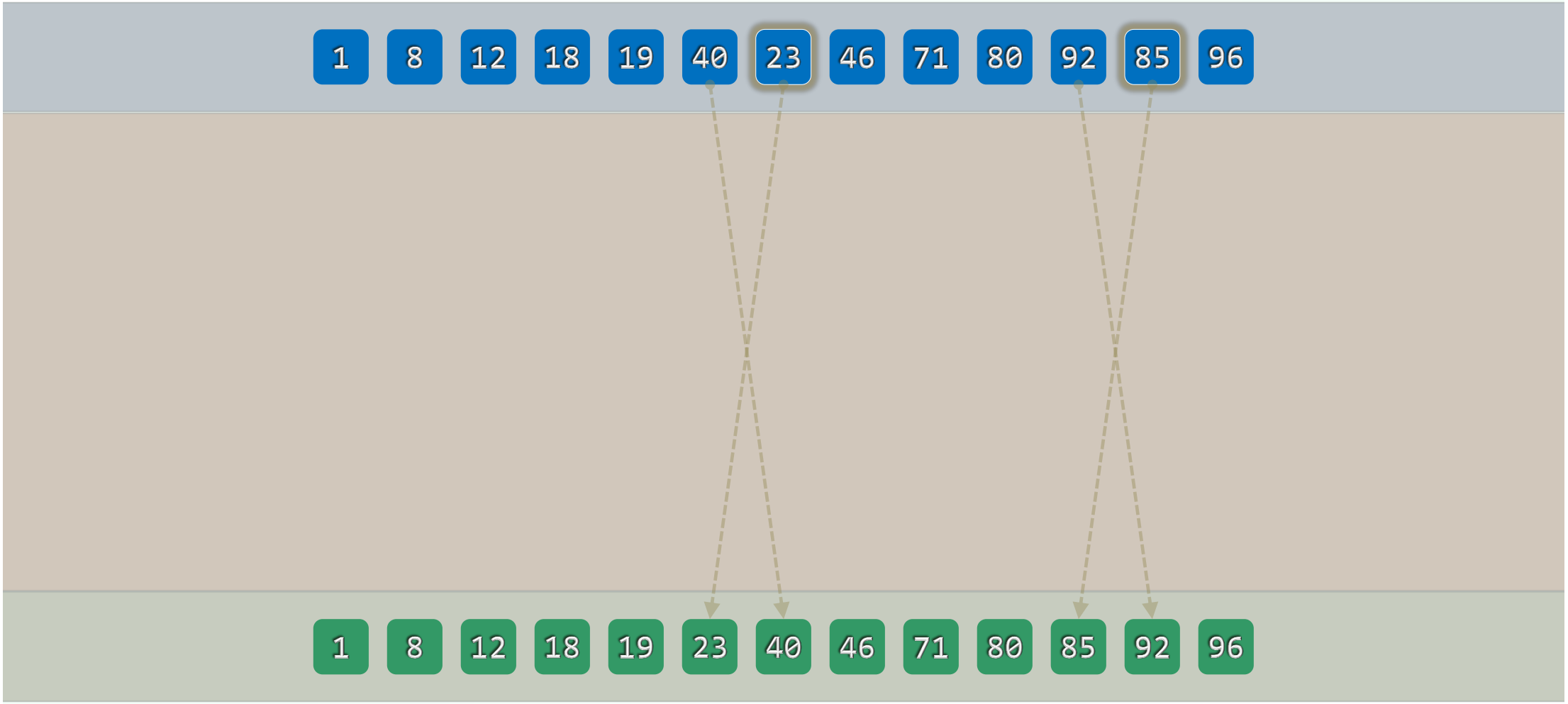
实例：$h_3 = 3$


1 8 19 40 12 71 18 85 80 23 96 46 92
1 40 18 23 92
8 12 85 96
19 71 80 46
1 18 23 40 92
8 12 85 96
19 46 71 80
1 8 19 18 12 46 23 85 71 40 96 80 92

实例：$h_2 = 2$


1 8 12 18 19 40 23 46 71 80 92 85 96
1 19 12 23 71 96 92
8 18 46 85 40 80
1 12 19 23 71 92 96
8 18 40 46 80 85
1 8 19 18 12 46 23 85 71 40 96 80 92

实例：$h_1 = 1$

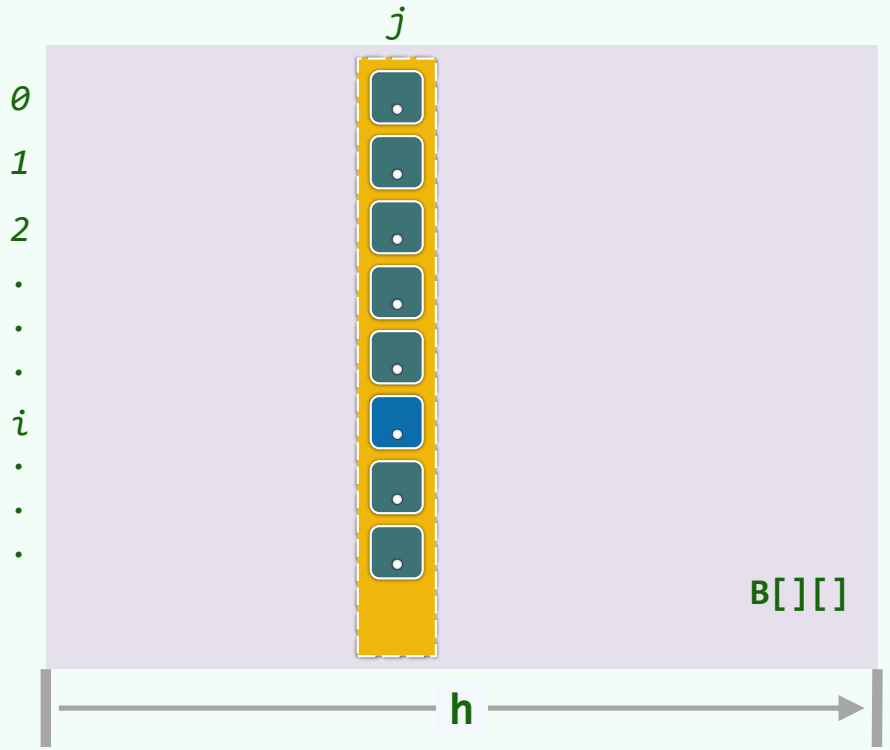


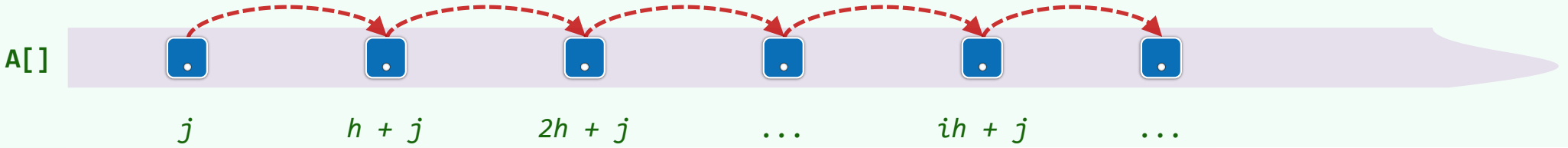
Call-by-rank

- 如何实现矩阵重排？莫非，需要使用二维向量？
- 实际上，借助一维向量足矣
- 在每步迭代中，若当前的矩阵宽度为h，则

$$B[i][j] = A[i \cdot h + j]$$

或 $A[k] = B[k/h][k\%h]$





实现

```
template <typename T> void Vector<T>::shellSort( Rank lo, Rank hi ) {
   // Using PS Sequence { 1, 3, 7, 15, 31, 63, 127, ..., 1073741823, ... }
   for ( Rank d = 0x3FFFFFFF; 0 < d; d >>= 1 )
      for ( Rank j = lo + d; j < hi; j++ ) { //for each j in [lo+d, hi)
         T x = _elem[j]; Rank i = j - d;
         while ( lo <= i && _elem[i] > x )
            { _elem[i + d] = _elem[i]; i -= d; }
         _elem[i + d] = x; //insert [j] into its subsequence
      }
} //0 <= lo < hi <= size <= 2^30
```

排序
希尔排序：Shell序列+输入敏感性

Shell's Sequence

- Shell 1959: $\mathcal{H}_{shell} = \{1, 2, 4, 8, 16, 32, 64, \ldots, 2^k, \ldots\}$
- 实际上，采用 $\mathcal{H}_{shell}$，在最坏情况下需要运行 $\Omega(n^2)$ 时间...
- 考查由子序列 A = unsort[0, 2^{N-1}) 和 B = unsort[2^{N-1}, 2^N) 交错而成的序列

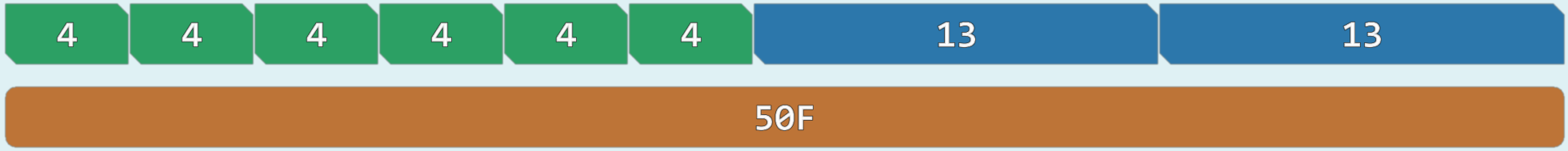
- 在做2-sorting时，A、B各成一列；故此后必然各自有序

- 然而其中的逆序对依然很多，最后的1-sorting仍需 $1 + 2 + 3 + \cdots + 2^{N-1} = \Omega(n^2/4)$ 时间
- 根源在于，$\mathcal{H}_{shell}$中各项并不互素，甚至相邻项也非互素

Postage Problem

- The postage for a letter is **50F**, and a postcard **35F**

  But there are only stamps of **4F** and **13F** available

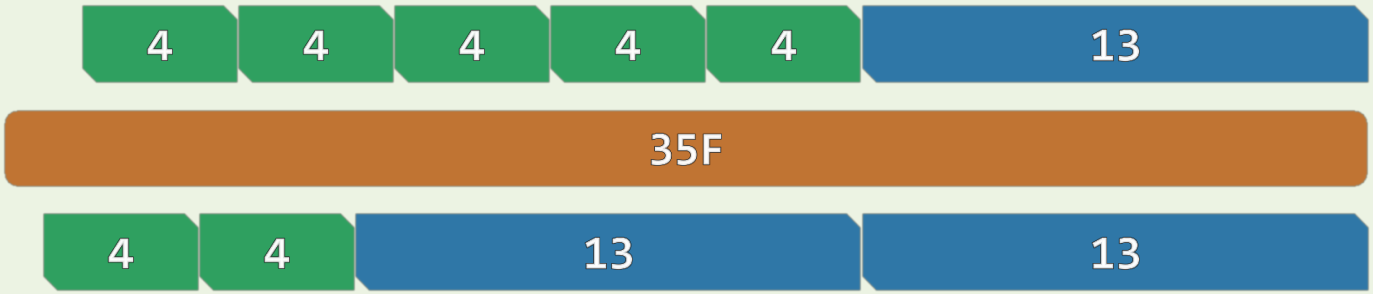


- Possible to stamp

  the letter and

  the postcard

  EXACTLY?



- Given a postage P, determine whether $P \in \{ n \cdot 4 + m \cdot 13 \mid n, m \in \mathcal{N} \}$

Linear Combination

- Let $g, h \in \mathcal{N}$
- For any $n, m \in \mathcal{N}$, $n \cdot g + m \cdot h$ is called a **linear combination** of g and h
- Denote $\mathbf{C}(g,h) = \{\, ng + mh \mid n, m \in \mathcal{N} \,\}$

  $\mathbf{N}(g,h) = \mathcal{N} \backslash \mathbf{C}(g,h)$ //numbers that are **NOT** combinations of g and h

  $\mathbf{x}(g,h) = \max\{\, \mathbf{N}(g,h) \,\}$ //always exists?
- Theorem: when g and h are **relatively prime**, we have

  $\mathbf{x}(g,h) = (g-1)\cdot(h-1) - 1 = gh - g - h$

  e.g. $\mathbf{x}(3,7) = 11, \quad \mathbf{x}(4,9) = 23, \quad \mathbf{x}(\boxed{4}, \boxed{13}) = \boxed{35}, \quad \mathbf{x}(5,14) = 51$

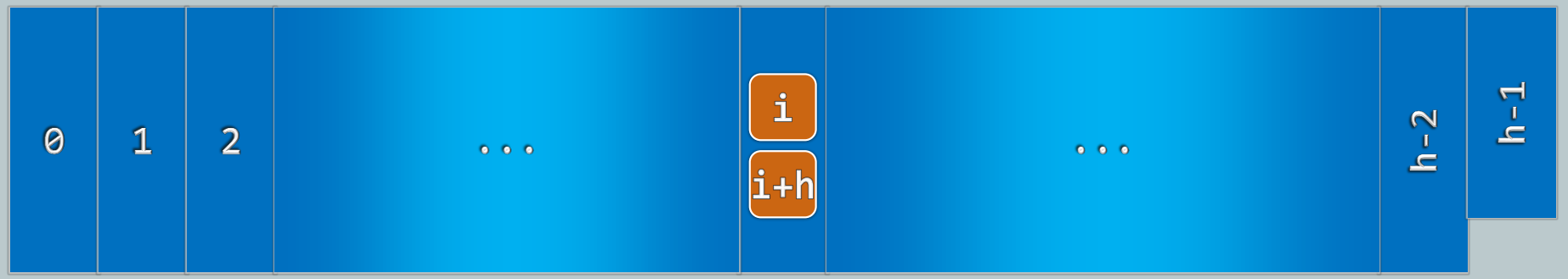
h-sorting & h-ordered

❖ A sequence S[0,n) is called h-ordered if $S[i] \leq S[i+h],\ \forall\ 0 \leq i < n-h$

❖ A 1-ordered sequence is sorted

❖ h-sorting: an h-ordered sequence is obtained by

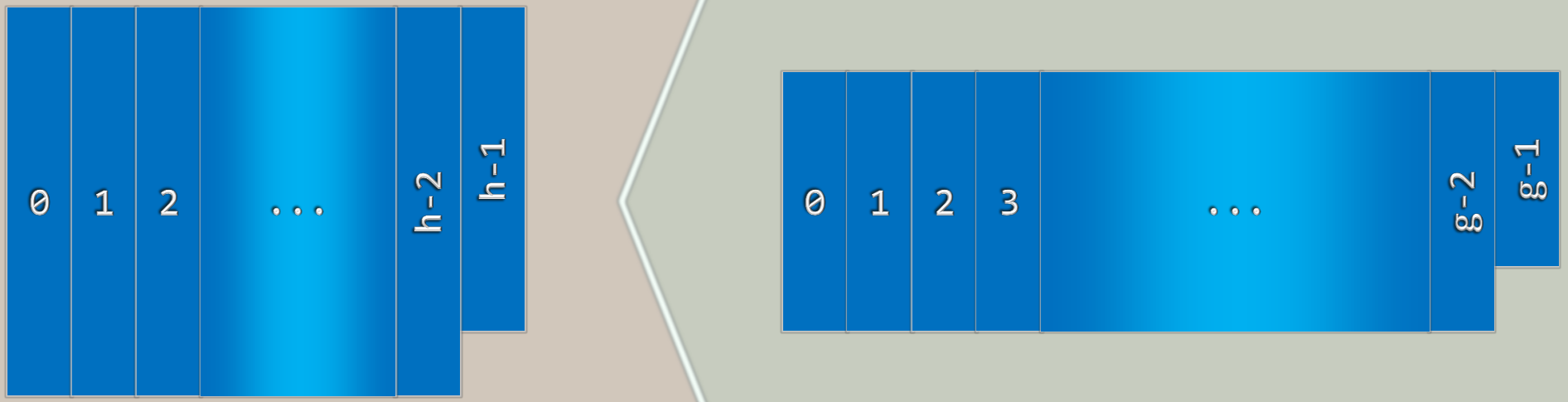
- arranging S into a 2D matrix with h columns and
- sorting each column respectively



Theorem K

❖ [Knuth, ACP Vol.3 p.90] //习题解析[12-12, 12-13]

A g-ordered sequence REMAINS g-ordered after being h-sorted.

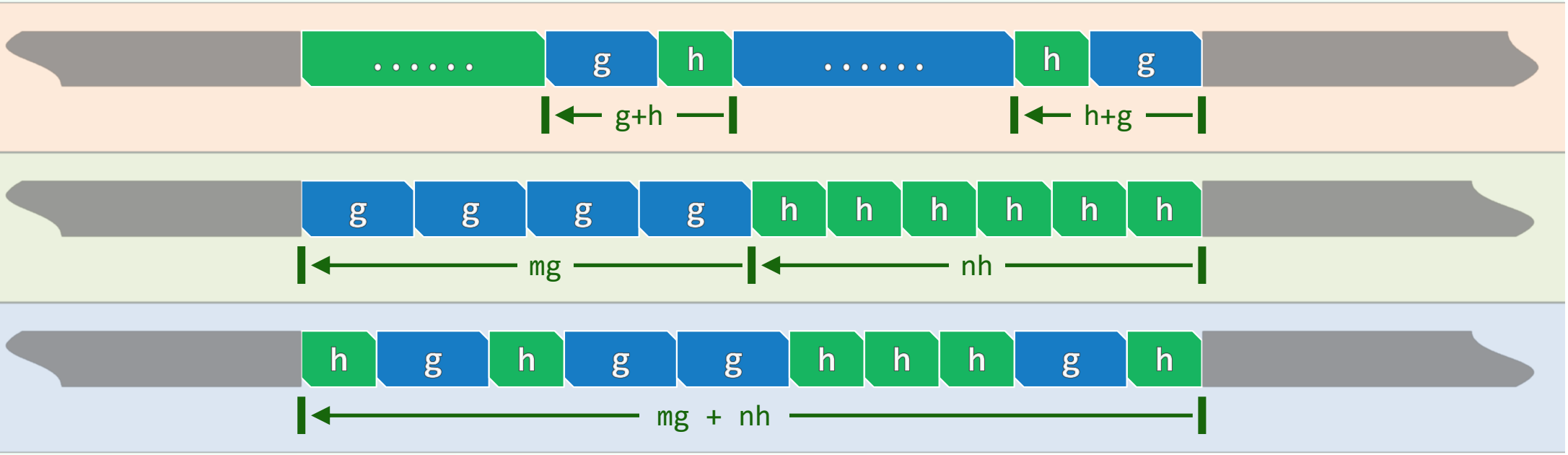


Linear Combination

A sequence that is both g-ordered and h-ordered

is called (g,h)-ordered, which must be both

(g+h)-ordered and (mg+nh)-ordered for any m, n ∈ N



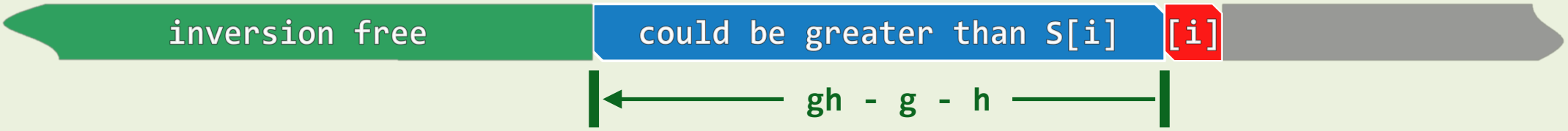
Inversion

- Let S[0,n) be a (g,h)-ordered sequence, where g and h are relatively prime
- Then for all elements S[j] and S[i], we have

$$i - j > \mathbf{x}(g,h) \qquad \text{only if} \qquad S[j] \leq S[i]$$

- This implies that to the LEFT of each element,

  only the previous $\mathbf{x}(g,h)$ elements could be GREATER



- There would be no more than $n \cdot \mathbf{x}(g,h)$ INVERSIONs altogether